Self-healing CI for verified Rust

Approach, data and early results

Dana Mukusheva · Beneficial AI Foundation · November 2026

01

Data

Where proofs break, and how often

Proofs break when the code under them moves

Rust change
A function changes, so its extracted Lean changes

Aeneas / Charon update
Same Rust, different extracted Lean

Lean / Mathlib bump
Tactics and lemma names change

15 / 39

changes to curve25519-dalek-lean-verify's extracted code or pins broke proofs

7 / 15

needed a theorem statement change, escalated to a human decision. E.g., PR 535 weakened the *_clamped specs (issue 873).

Source: curve25519-dalek-lean-verify, whole history, Oct 2025 – Sep 2026 (331 commits).

How we find real breaks

1 · MINE

39

candidates: commits that change extracted Lean or Lean/Aeneas pins. Read from git, no build.

2 · FILTER

15

likely breaks: a dependency changed and humans then edited proofs

3 · REPLAY

15 / 17

confirmed: build the old proofs on the new extracted code and pins. The human fix is the ground truth.

4 · CLASSIFY

5 proof repair
7 statement change
3 build / imports

Class	What failed in the replay	Bot's job
Proof repair	Only proofs fail; every statement still type-checks	Repair, gate decides
Statement change	A theorem statement no longer type-checks	Escalate to a human
Build / imports	A module or import moved, or a duplicate declaration	Structural fix, V1

curve25519-dalek-lean-verify. 17 replayed = 15 likely + 2 controls. Replay vs. the human fix: recall 0.99, precision 0.83.

The same method on other repos

Repo	Candidates	Replayed	Breaks	Classes
dalek-lean-verify	39	17	15	5 proof · 7 statement · 3 build
SPQR (Signal)	22	10	10	5 proof · 3 statement · 2 build
dalek-u32	3	0	0	No likely breaks yet
SymCrypt (Microsoft)	3	–	–	No break history: generated below

SymCrypt: upgrade its Aeneas step by step, re-extract, see what breaks

Aeneas step	What broke	Class
6e167c9b · Jul 24	3 proofs	Proof repair
5d08da45 · Aug 19	Extraction itself fails	Stop, report the tool error
12a018bb · Sep 22	228 proofs + 21 statements in models	Mostly escalate
557f7a15, cd419953	Lean module-system migration	Structural fix

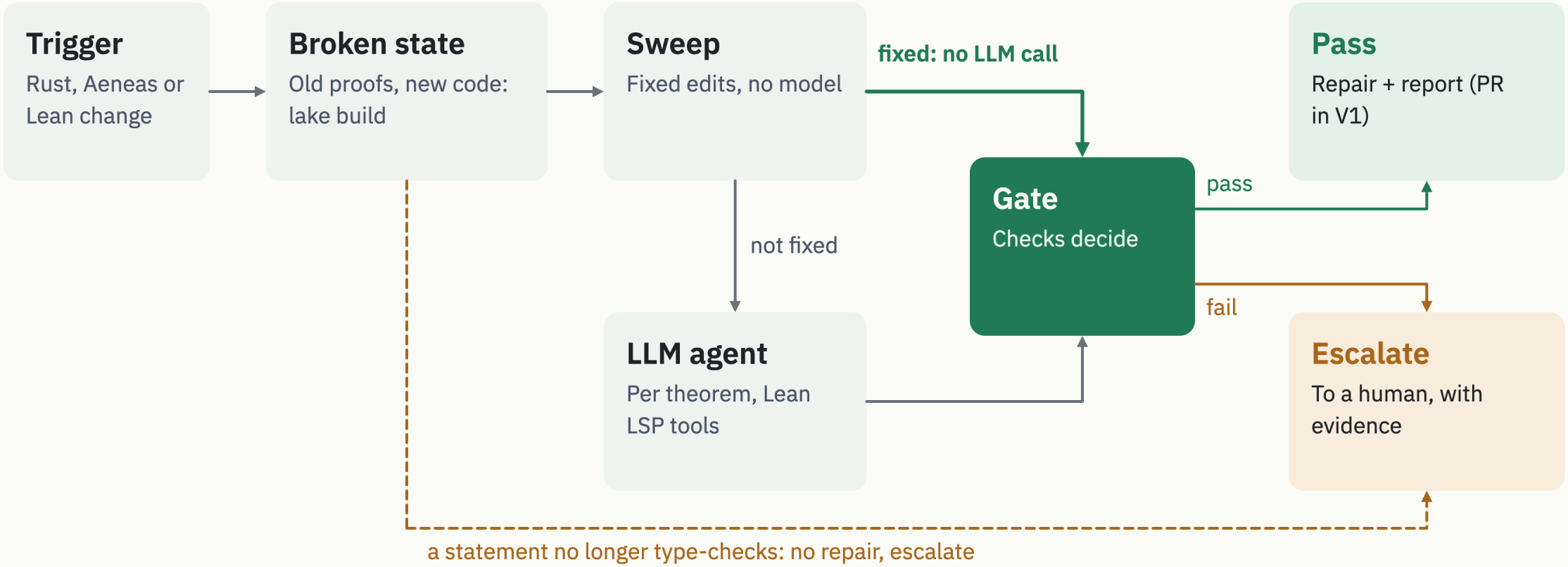
Whole main-branch history, mined Oct 2026. SymCrypt cases are held out: never seen while building the bot.

02

Method

The pipeline, the gate and the CLI

LLMs propose, scripts enforce



The bot never edits a theorem statement and never decides whether a spec change is acceptable.

The sweep: cheap fixes before any model

Lean error	What it means	Sweep's fix
no goals	The proof now ends early; a tactic is left over	Delete the leftover tactic
heartbeats	A step hit Lean's time limit	maxHeartbeats 1000000 (5×), shown in the report
unsolved goals	Goals remain at the end of the proof	Append the standard closer

Standard closer

```
all_goals first | omega | simp_all | decide | rfl
```

Tried in order on each open goal: omega (linear arithmetic), simp_all (simplifier on goal and hypotheses), decide (compute it), rfl (true by definition).

Future improvement

- grind: Lean's SMT-style automation
- aesop: rule-based proof search
- Aeneas tactics: scalar_tac, step*
- Recipes learned from accepted repairs

Which would you add? Advice welcome.

Up to 3 fixes per theorem, then one build to check the result. Whatever the sweep produces still goes through the full gate.

The gate is the trusted core

Check	What it enforces
G0 Scope	Only broken proofs change; no new axiom, sorry or import
G1 Statements	Statement fingerprints unchanged (StmtCanon, by Liao)
G2 Trust base	Axioms ⊆ Lean's standard 3 + what the theorem used before. Inherited sorry: accepted, flagged
G3 Clean build	Fresh checkout builds; nothing else newly fails
G4 Quality (V1)	Report only, never blocks: proof size and build time vs. the original, so reviewers spot bloated or slow repairs
G5 Kernel replay	Built proofs re-checked by the Lean kernel (lean4checker, Lean FRO)
Audit	Flags statement changes made by the change itself

Why trust the gate?

Says no when unsure

Missing or incomplete input: gate-unavailable, never pass

Tested against cheating

Planted statement edits, sorry, axioms, out-of-scope files: all rejected

Next step: formally verify the gate's code

Prove: accepted ⇒ statements unchanged

Threat model: honest mistakes and obvious shortcuts, not adversarial proofs.

Try it: the CLI

`uv run shfv doctor`	Check Lean, the agent, the MCP server, the target clone
`uv run shfv repair --case dalek-lean-verify/e65b46d0`	Replay one historical break end to end (PR 782)
`uv run shfv repair --case … --model claude-sonnet-5-5`	Same case, another model
`uv run shfv render runs/<case>/latest/report.json`	Re-render the report the bot would post on the PR
`uv run gate compare --config T --base B --broken K --candidate C --diff D`	The gate on its own: accept or reject a candidate
`uv run gate audit --config T --base B --head H`	Statement changes made by the change itself

```
03:06 [1/1] to_radix_2w_size_hint_spec → agent attempt 1 (claude-code, claude-opus-5-5)
09:10 → scope ✓ · clean rebuild ✓ · statements ✓ · no new axioms/sorry ✓ · lean4checker ✓
ACCEPTED · repaired 1 · $0.28 · 9m10s → runs/…/report.md
```

What the PR sees: a repair

self-healing-fv bot commented on PR #782 “Chore: update Aeneas”

Proofs repaired: 1 of 1 broken proof fixed, accepted by the gate

Cause: Aeneas 28ad838 → 42c0e90 · **Broken:** 1 proof, 0 statements · **Cost:** $0, no agent run · 8m28s

Gate: G0 scope ✓ · G1 1328 statements unchanged ✓ · G2 no new axioms or sorry ✓ · G3 clean rebuild ✓ · G5 lean4checker ✓

Audit of the change itself: no statement, definition or trust assumption changed

```
· step (config := { maxSteps := 1 })
step*
- have : w.val = 8:= by scalar_tac
- simp_all
```

The proposed diff: leftover tactics deleted (here 2 of 13 lines). Collapsed below it: the human fix, for comparison.

Abridged from a real run on the scoreboard commit. The full comment is 109 lines.

What the PR sees: an escalation

self-healing-fv bot commented on PR #74 “chore: update aeneas”

⚠ Needs a human: 1 theorem statement no longer type-checks

Cause: Aeneas d6ff318 → 9aaaed2 · **Broken:** 0 proofs, 1 statement · **Cost:** $0, no agent run · 3m27s

Escalated: sub_loop_spec · statement-must-change. Lean: argument *a* has type Array U64 5 but is expected to have type U64 in *sub_loop a*

```
What changed in the extracted code:
- sub_loop (a b difference : Scalar52) (mask borrow : U64) (i : Usize) : Result Scalar52
+ sub_loop (mask : U64) (a b difference : Scalar52) (borrow : U64) (i : Usize)
: Result (Scalar52 × U64)
```

The current statement is shown unchanged. Someone who owns the spec decides what it should now promise.

Abridged from a real run on the scoreboard commit. In history, the human rewrote the statement and left a sorry.

03

Configurability

What each repo controls, and what it can’t

What users control

Agent

- v0: Claude Code, headless
- Next: test Codex CLI and Factory Droid
- Any other agent on request: one adapter + config

Model

- Any model the agent supports
- Tested: Opus 5.5, Sonnet 5.5
- Self-hosted open models, e.g. via Droid

Budget

- Per theorem: $, minutes, turns, attempts
- Per run: $ and one time deadline
- $ from tokens × a per-model price table

Tools

- Profile: compiler only, or Lean LSP
- Shell: lake build, lake env lean
- No web; an agent that can’t comply won’t start

Repair strategy

- Automation sweep on or off
- Attempts per theorem, retry with feedback
- Later: sweep tactics, cheap-to-strong model cascade

Repo and scope

- Build targets and modules to audit
- Protected: extracted code, trusted models, build config
- Runs: local CLI now, GitHub Actions next

One TOML file per target repo. Not configurable: the gate’s acceptance rules.

04

Performance

Early results on real breaks

Scoreboard: 15 dalek breaks + 1 SymCrypt

⚠ **Too few dalek cases to measure proof-body repair.** Only 3 of 16 needed the agent, and 2 of those carry flags. Held-out SPQR adds 4 (next slide).

Outcome	Cases	Which
Accepted, fixed by the sweep (no model)	2	PRs 41, 782 · $0
Accepted, fixed by the agent	3	PR 278 $1.21 🏷 · PR 734 $5.21 🏷 · SymCrypt $0.97
Escalated: a statement no longer type-checks	9	PRs 74, 101, 232, 343, 535, 584, 638, 764, 865 · $0. On PR 535 the audit flags the weakened *_clamped specs
Escalated: broken state can't be checked	2	PRs 720, 824 · setup limit, not the agent

16 of 16 decided

Each run ended accepted or escalated, none crashed. Agent cost $7.39 in total

Since then

Held-out SPQR, the human-fix comparison, a second agent and GitHub Actions: next slides. Living numbers: docs/RESULTS.md

All 15 confirmed dalek breaks + 1 generated SymCrypt case · tool 20f70af · Opus 5.5 · 🏷 rests on lemmas already unproved at the base · PRs 41, 782 were development cases · costs are API list-price estimates

Held-out SPQR: 4 of 5 cases accepted

PR	Outcome	Proofs	Cost	Compared with the human fix
359	✅ accepted	1 / 1	$0.74	Bot added a helper lemma; human edited one unfold
493	✅ accepted	3 / 3	$0.89	Bot fixed renamed hypotheses; human used sturdier forms
52	✅ accepted	13 / 13	$2.35	1 edit identical; human also strengthened one statement
679	✅ accepted	15 / 15	$6.34	1 edit identical; one helper lemma close to the human's (0.82)
106	⚠️ escalated	0 / 3	$0	Ineligible: Aeneas renamed axioms in a protected file

32 of 32, no flags

Every acceptance passed the full kernel replay and rests on no inherited sorry

$10.31

Agent cost for all 5 cases, 54 minutes in total. 31 proofs repaired by the agent, 1 by the sweep

Held out: never used while building the bot · one run each · tool ef44fbd · Opus 5.5 · SPQR is public: PR 52 (May 2026) predates the model's ~June 2026 training cutoff, the other 3 accepted cases (19 proofs) come after it

Compared with the human fix

dalek theorems, by what the bot did and what the human did to the statement

Bot	Human kept it	Human changed it
Repaired	17	0
Escalated	13	36
Not attempted (ineligible case)	53	23

Removed theorems and definitions left out (12)

0: never re-proved a changed spec
Every repair agrees with the human on the statement

13: gate too strict
11 needed only namespace/open edits

23: still type-checks, changed anyway
“Type-checks” ≠ “spec still right”

shfv compare-human on the dalek scoreboard runs (tool 20f70af) · proofs: 3 of 10 comparable agent edits identical to the human’s, all 1–2 lines; median similarity 0.50; similar size (55 vs 68 changed lines)

Two agents, same outcomes, different proofs

	Codex (GPT-6.1-Sol)	Claude Code (Opus 5.5)
Proofs accepted	49 of 49	49 of 49
Agent time	63 min	64 min
Output tokens	89k	213k
Cost	at most $10.56	$18.27
Identical to the human edit	4 of 38	4 of 38

8 of 44

edits identical between the two agents. The gate judges the result, so the agent is swappable

Opposite routes, both accepted

PR 734 to_bytes_spec: Claude raised maxHeartbeats; Codex rewrote 347 lines to fit the limit

7 agent cases (dalek PRs 278, 734 · SymCrypt · SPQR PRs 52, 359, 493, 679) · one run each, same tool commit fe5aaff · not a ranking: both repaired everything · Codex adapter written by Codex, 2 lines of shared code changed · Codex cost: upper bound from OpenAI's published prices

Runs on GitHub Actions

	Run 1: dalek PR 782	Run 2: dalek PR 278
Fixed by	sweep, no model	agent (Claude Code, repo's key)
Result	✅ accepted	✅ 6 of 6 accepted 🏷️
Whole run	56 min	31 min
Model cost	$0	$1.03 (cap $3)
Runner cost	~$1.25	~$0.75

8-core larger runner

The standard 2-core runner timed out after ~2 h rebuilding the case

Key never exposed

Repo secret; Claude Code strips it from the agent's commands (needs bubblewrap on Linux). Not in logs or artifacts

Build cache: −34% setup

After veritooling's CI caching: case setup 859 → 568 s, restored in 5 s. Repeat runs: 6 of 6 again, tripwire clean

Manually dispatched replays (workflow_dispatch), 2026-10-27/28 · runner by Alessandro · 🏷️ rests on lemmas already unproved at the base, as in local runs · runner cost: billed minutes, estimate

From replay to live PRs

Live run	Outcome	Cost	Time
libsignal-verify PR 17	✅ 2 of 2 accepted	$0.36	4 min
dalek PR 278	✅ 6 of 6 accepted 🏷️	$1.24	14 min
SPQR Aeneas bump (dry run)	⚠️ trusted model needs a rename; after it, 0 proofs broke	$0	7 min
libsignal-verify Lean 4.35 bump (dry run)	⚠️ 17 statements broken by the VCVio API, for a human. Libsignal half: ✅ 1 of 1	$0.39	6–8 min

Gate judges only the bot's edits

Against the PR head. The PR's own statement and trust changes are listed for the reviewer. An independent review found holes in 3 rounds, all fixed

Runs from the target repo

Live on libsignal-verify since Nov 3, after an end-to-end test on a fork. One comment per PR; pushes nothing; a label or repo variable turns it off

shfv repair --base --head on the final gate (self-healing-fv #18–#22), 2026-10-29 – 11-03 · dry runs local, nothing pushed · 🏷️ rests on lemmas already unproved at the base

Found while testing

Replays: the answer is on disk

In a replay, the human fix is on the same machine (the fixed tree, and the target repo's git history), and Claude Code and Codex restrict writes, not reads. Transcript scan of all runs: 1,671 tool calls, none read the human fix.

A measurement problem, not the bot's: live runs have no human fix. **Fix:** every report flags reads outside the workspace (detect, then re-run). A read sandbox comes later, for security on untrusted PRs.

Gate too strict in two places

11 of 13 "statement no longer type-checks" escalations needed only open/namespace edits, outside any proof. SPQR PR 106: Aeneas renamed axioms in a protected file.

Live runs now list the change's own trust edits instead of blocking. Open/namespace edits: still an owner decision.

Type-checking misses spec changes

In 23 of 95 dalek proof breaks, the human changed a statement that still type-checked. A bot would re-prove the old spec.

V1 adds a "spec suspect" outcome for these.

Sources: human-fix comparison on the dalek scoreboard (shfv compare-human), held-out SPQR runs, Codex adapter review

Status and next steps

Where we are

- Done: scoreboard, held-out SPQR, human-fix comparison, Codex vs Claude, GitHub Actions with the agent
- Done: live mode on real PRs, tested end to end on a fork
- Live: shadow mode on libsignal-verify since Nov 3; SPQR next

Next

- Partial acceptance: repair what doesn’t depend on a broken statement
- Scheduled Aeneas bump PRs, a spec-suspect outcome, verify the gate
- For bump authors: push with statements fixed, proofs still broken
- Welcome: repos and breaks to add to the dataset

Thanks to Liao (StmtCanon, harness design), Alessandro (swvf, direction, CI runner), Oliver and Jure (SPQR pilot), Lara (veritooling) and Shaowei.